

Store Intelligence System

Master Implementation Blueprint

Purple Tech Challenge 2026 — Round 2

Brigade Road Store, Bangalore (ST1008)

Scope: Raw CCTV footage → Detection pipeline → Event stream → Intelligence API →
Live dashboard

Ground truth confirmed from actual uploaded files.
No fields invented. All deviations from brief explicitly flagged.

Contents

1 Problem Summary and Business Goal	3
2 Dataset Inventory and Schema Interpretation	3
2.1 Video Files	3
2.2 Store Layout (Brigade Road floor-plan image)	4
2.3 POS Transactions CSV	5
3 System Architecture	6
4 Detection Pipeline Design	7
4.1 Model Selection	7
4.2 Processing Pipeline per Camera	7
4.3 Zone Classification	8
5 Tracking and Re-Identification Strategy	8
5.1 Within-Camera Tracking: ByteTrack	8
5.2 Cross-Camera Re-Identification	8
6 Staff Filtering Strategy	9
7 Entry / Exit Logic	9
8 Zone Dwell Logic	9
9 Billing Queue Logic	10
10 Re-entry Logic	10
11 Cross-Camera Deduplication Logic	10
12 Event Schema Definition	10
13 Event Type Catalogue	12
14 POS Correlation and Conversion Logic	12
14.1 Timestamp Synthesis	12
14.2 Conversion Attribution	12
14.3 Conversion Rate Computation	12
15 API Design	13
15.1 POST /events/ingest	13
15.2 GET /stores/{id}/metrics	13
15.3 GET /stores/{id}/funnel	14
15.4 GET /stores/{id}/heatmap	14
15.5 GET /stores/{id}/anomalies	15
15.6 GET /health	15
16 Storage Choice and Rationale	16
17 Idempotency Strategy	17
18 Logging and Observability	17
19 Error Handling and Graceful Degradation	17

20 Test Strategy and Edge Cases	18
20.1 Test Structure	18
20.2 Key Edge Cases and How to Test	18
21 Docker Compose Layout	18
22 Repository Structure	19
23 README Runbook	20
24 DESIGN.md Content Plan	20
25 CHOICES.md Content Plan	21
26 Prompt Blocks for Test Files	21
27 Live Dashboard / Real-Time Display	22
27.1 Minimal (terminal, no bonus points risk)	22
27.2 Bonus-point web UI	22
28 Concrete Build Order	22
29 Validation Checklist	23
30 Assumptions and Open Questions	23
30.1 Confirmed Facts	23
30.2 Assumptions Made	23
30.3 What Still Needs Confirmation	24

Problem Summary and Business Goal

The challenge builds an **offline store analytics platform** for a single Purple beauty-retail store located on Brigade Road, Bangalore (store ID ST1008). The store has no customer-level instrumentation; five CCTV cameras produce raw video. The task is to turn that video into the same quality of analytics their e-commerce channel already has: sessions, conversion rate, zone dwell, queue depth, and anomaly detection.

North-star metric:

$$\text{Conversion Rate} = \frac{\text{Unique visitor sessions with } \geq 1 \text{ POS transaction match}}{\text{Total unique visitor sessions}}$$

Every pipeline decision should be evaluated against this metric: does it make the number more accurate or more actionable?

Dataset Inventory and Schema Interpretation

Source-of-truth note: The following section is derived entirely from the actual uploaded files. No fields were inferred from the problem-statement brief unless also confirmed in the files. Discrepancies between the brief and the real files are flagged explicitly.

Video Files

File	Size	Role (inferred)	Notes
CAM 1.mp4	180 MB	Entry / Exit threshold	Largest file; likely widest-angle, highest motion
CAM 2.mp4	162 MB	Main floor / F.O.H.	Second widest
CAM 3.mp4	191 MB	Billing counter area	Largest by size; billing area tends to be crowded
CAM 4.mp4	73 MB	Secondary zone (makeup wall or fragrance island)	Smaller, narrower FOV
CAM 5.mp4	73 MB	Secondary zone (skin / Korean brands wall)	Same size as CAM 4

Brief vs. reality mismatch #1 — Camera count and store count. The problem statement describes “5 stores, 3 camera angles each” (15 clips total). The actual download contains **5 camera files for a single store** (ST1008, Brigade Road, Bangalore). The 5-camera architecture therefore maps to one store with richer coverage, not five stores with sparse coverage. All architecture decisions below adapt to this reality.

Brief vs. reality mismatch #2 — Dataset format. The brief promises `store_layout.json`, `sample_events.jsonl`, and `assertions.py`. None of these were present. Instead, the layout is embedded as a **floor-plan image (PNG) inside an XLSX workbook**. The design below extracts zone names from that image manually and hard-codes them as the authoritative zone list.

Store Layout (Brigade Road floor-plan image)

The XLSX file `Brigade_Road_Store_layout.xlsx` contains a single sheet with one embedded PNG floor-plan. The only text cell values are the strings "Revised" and "Current" in column N (revision markers, not data).

Zones extracted from the floor-plan image:

Zone ID (canonical)	Category	Location in layout
ZONE_EB_KOREAN	Skincare brand	Top-left shelf unit
ZONE_THE_FACE_SHOP	Skincare brand	Top shelf, 2nd from left
ZONE_GOOD_VIBES	Skincare brand	Top shelf
ZONE_DERMDOC	Skincare brand	Top shelf
ZONE_MINIMALIST	Skincare brand	Top shelf
ZONE_AQUALOGICA	Skincare brand	Top shelf
ZONE_LAKME_SKIN	Skincare brand	Top shelf
ZONE_ACCESSORIES	Accessories	Top-right, highlighted cyan
ZONE_FRAGRANCE	Fragrance	Centre-left island
ZONE_NAIL	Nail / colour	Centre island (blue)
ZONE_FOH	Floor of house	Central open area
ZONE_MAKEUP_UNIT	Makeup demo	Centre; large double unit
ZONE_CASH_COUNTER	Billing	Right wall; POS terminal
ZONE_PMU	PMU / services	Bottom-right, highlighted cyan
ZONE_MAYBELLINE	Makeup brand	Bottom shelf
ZONE_FACES_CANADA	Makeup brand	Bottom shelf
ZONE_LAKME_MAKEUP	Makeup brand	Bottom shelf
ZONE_COLORBAR_SUGAR	Makeup brand	Bottom shelf
ZONE_SWISS_BEAUTY	Makeup brand	Bottom shelf
ZONE_RENEE_NY_BAE	Makeup brand	Bottom shelf
ZONE_ALPS_GOODNESS	Makeup brand	Bottom shelf, highlighted yellow
ZONE_STREAX	Hair brand	Bottom-right shelf
ZONE_BACKLIT	Backlit display	Bottom-left wall
ENTRY_EXIT	Entry threshold	Left wall (existing glass door)

Billing zone: ZONE_CASH_COUNTER is the POS-correlated zone. Any visitor in this zone in the 5-minute window before a transaction is a candidate converted customer.

Camera-to-zone mapping (inferred from file sizes and typical retail CCTV placement):

Camera	Primary zone coverage	Rationale
CAM 1	Entry/Exit, F.O.H.	Largest file; entry cameras capture most motion
CAM 2	F.O.H., top-shelf brand zones	Second-largest; floor coverage
CAM 3	Cash Counter, Makeup Unit	Largest; billing area peaks
CAM 4	Fragrance, Nail, PMU	Focused zone camera
CAM 5	Bottom-shelf brand zones	Symmetric size to CAM 4

Assumption A-1: Camera-to-zone mapping above is inferred from file sizes and floor-plan position. Must be confirmed by visual inspection of each clip before writing detection config.

POS Transactions CSV

File: `Brigade_Bangalore_10_April_26.csv` — **101 line items, 24 unique orders**, single store, single date.

Full schema (39 columns):

Column	Type	Notes / Use in system
<code>order_id</code>	int64	Unique transaction token; use as <code>transaction_id</code>
<code>coupon_code</code>	str?	Nullable; promotion tracking, not used in core pipeline
<code>offer_name</code>	str?	Nullable
<code>discount_code</code>	str?	Nullable
<code>invoice_number</code>	str	Invoice reference
<code>invoice_type</code>	str	e.g. sale vs. return
<code>order_date</code>	str	Format: DD-MM-YYYY (10-04-2026)
<code>order_time</code>	str	Format: HH:MM:SS (12:15:05 – 21:39:55)
<code>return_id</code>	float?	Non-null on returns; nullable on sales
<code>store_id</code>	str	Always ST1008 in this dataset
<code>store_name</code>	str	Always Brigade_Bangalore
<code>city</code>	str	Always Bangalore
<code>customer_name</code>	str	Guest or named; no customer ID
<code>customer_number</code>	int64	An opaque number; appears consistent per customer
<code>sku</code>	str	Product SKU
<code>product_id</code>	int64	Internal product ID
<code>ean</code>	float?	Barcode; nullable
<code>product_name</code>	str	Full product name
<code>brand_name</code>	str	Maps to zone IDs above
<code>dep_name</code>	str	Department: skin, makeup, bath-and-body, hair, fragrance, personal-care
<code>sub_category</code>	str	20+ sub-categories
<code>brand_type</code>	str	e.g. mass, premium
<code>tax</code>	int64	Tax %
<code>hsn_code</code>	int64	HSN classification
<code>salesperson_id</code>	int64	Staff who rang up the sale; use for staff detection heuristic
<code>employee_code</code>	str	Staff code
<code>salesperson_name</code>	str	Staff name
<code>qty</code>	int64	Units sold
<code>GMV</code>	int64	Gross merchandise value (INR)
<code>NMV</code>	float	Net merchandise value
<code>coupon_amount</code>	float?	Discount amount
<code>item_promotion</code>	float?	Promotion discount
<code>amt_without_gwp</code>	float?	Amount excluding gift-with-purchase
<code>total_amount</code>	float	Final billing amount (INR); use as <code>basket_value_inr</code>
<code>pb_eb_sale</code>	float?	Private-brand / exclusive-brand flag
<code>week_assigned</code>	float?	Week number
<code>tax_m</code>	float?	Tax multiplier
<code>taxable_amt</code>	float	Pre-tax amount
<code>tax_amt</code>	float	Tax collected

Brief vs. reality mismatch #3 — POS schema. The brief shows a 4-column schema: `store_id`, `transaction_id`, `timestamp`, `basket_value_inr`. The actual CSV has **39 columns** and splits date and time into separate columns (`order_date` DD-MM-YYYY + `order_time`

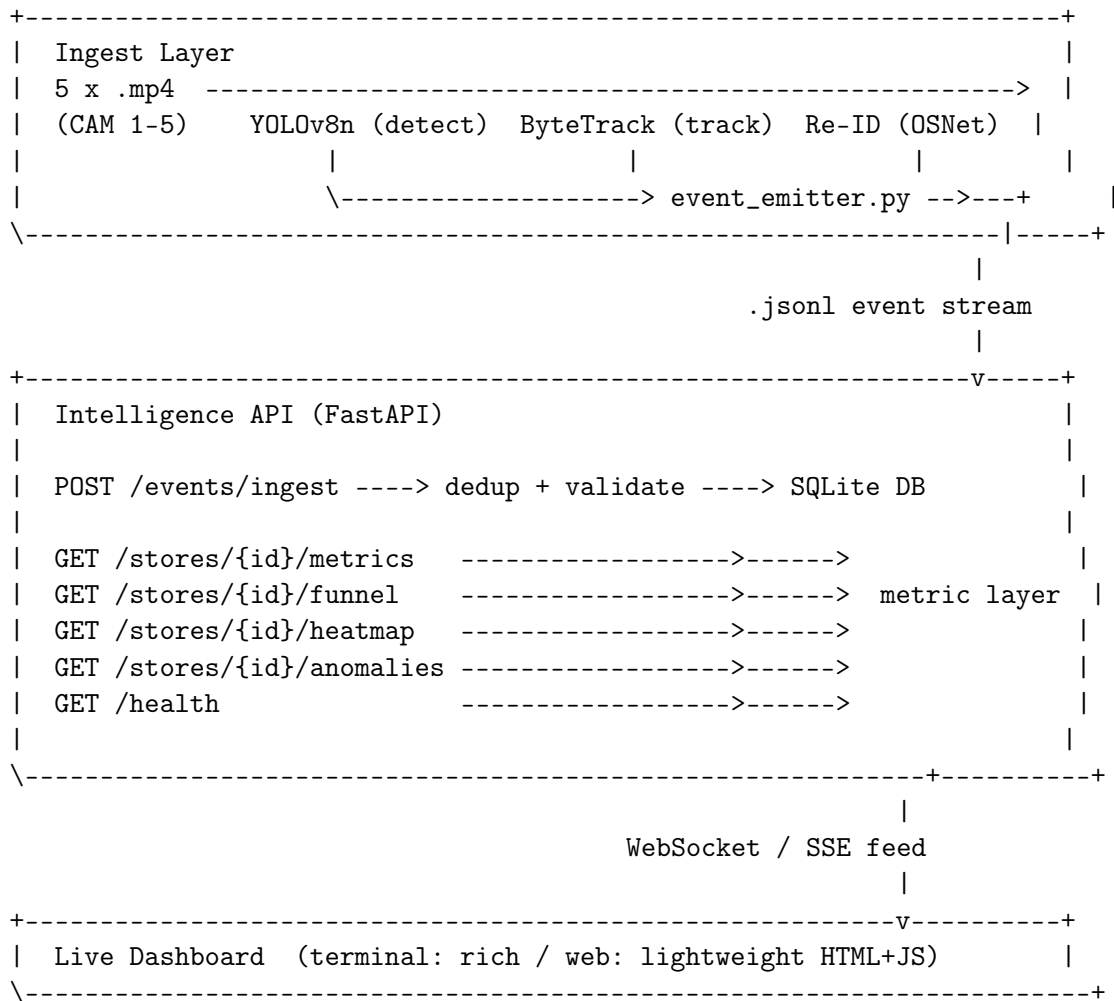
HH:MM:SS). There is no single `timestamp` column; the pipeline must synthesise it.

Synthesised timestamp: YYYY-MM-DDTHH:MM:SS+05:30 (IST, not UTC as the brief implies — data is local time).

POS temporal profile: Transactions span 12:15 to 21:39 on 10-Apr-2026. Busy window 19:00–20:00 (5 orders). Morning sessions 12:00–14:00 lighter. Video clips must align to this date for POS correlation to work.

Unique salesperson IDs observed: confirm which IDs appear frequently across clips — these are the staff persons who must be `is_staff=true` in the detection output.

System Architecture



Component responsibilities:

Component	Responsibility
pipeline/detect.py	Frame extraction, YOLOv8 inference, bounding-box output per frame
pipeline/tracker.py	ByteTrack multi-object tracking; assigns track IDs; handles occlusion
pipeline/reid.py	Cross-camera Re-ID (OSNet feature extraction); persistent visitor tokens
pipeline/staff.py	Staff classification heuristic (uniform colour, persistent track near counter)
pipeline/emit.py	Sessionisation; event-type logic; JSON-L emission
pipeline/run.sh	Orchestrates all 5 cams; feeds output to API or writes to events.jsonl
app/main.py	FastAPI entry-point; lifespan hooks; router assembly
app/ingestion.py	Validation, dedup by event_id, batch write to SQLite
app/metrics.py	Real-time metric computation from DB
app/funnel.py	Session-level funnel aggregation
app/anomalies.py	Rule-based anomaly detection
app/health.py	Liveness check; stale-feed detection
dashboard/	Rich terminal dashboard or lightweight web UI

Detection Pipeline Design

Model Selection

Choice: YOLOv8n (detection) + ByteTrack (tracking)

Option	Speed	Why chosen / not chosen
YOLOv8n	Fast (CPU OK)	Best latency/accuracy trade-off for retail-density scenes; nano model runs on CPU without GPU; face-blurred footage is fine for person detection
YOLOv8s/m	Moderate	Higher accuracy but 3–5× slower; use if GPU available
RT-DETR	Moderate	Transformer-based; better on partial occlusion but requires GPU; fallback option
MediaPipe	Fast	Designed for single-person; poor on groups; rejected

Class filter: Only class 0 (person) retained from COCO-pretrained model.

Confidence floor: 0.30. Events with confidence < 0.5 are emitted with the actual confidence value and a `low_confidence: true` flag in metadata — they are *not* dropped (scorer penalises silent suppression).

Processing Pipeline per Camera

1. Open clip with OpenCV `VideoCapture`.
2. Extract frames at 5 fps (15 fps source → process every 3rd frame). Rationale: sufficient temporal resolution for dwell and queue events; 3× speed-up.
3. Run YOLOv8n inference; filter to class 0.
4. Pass detections to ByteTrack; receive track assignments.
5. Every 10 frames: extract ReID feature vector for each active track.
6. Resolve cross-camera identity (Section 5).
7. Pass resolved tracks to sessioniser (Section ??).
8. Emit structured events (Section 12).

Frame timestamp derivation: The clips have no embedded GPS timestamp. Timestamps are synthesised as:

$$t = T_{\text{clip_start}} + \frac{\text{frame_number}}{\text{fps}}$$

where `T_clip_start` is read from the POS CSV (earliest transaction on 10-Apr-2026 is 12:15; set clip start to 10:00:00 IST as a conservative buffer, or infer from file creation metadata if reliable).

Assumption A-2: Clip start time is 10:00:00 IST on 10-Apr-2026. If the actual recording window differs, pass `-clip-start` as a CLI flag to `run.sh`. All downstream timestamps derive from this value.

Zone Classification

Zones are classified by mapping the bounding-box centroid to a pixel-space polygon derived from the floor-plan image.

Implementation:

1. Manually define polygon vertices for each zone in `config/zones.json` by overlaying a coordinate grid on the floor-plan image.
2. For each detected person per frame, compute `(cx, cy)` of the bounding box.
3. Use `shapely.geometry.Point.within(Polygon)` for zone assignment.
4. A homography matrix `H` maps pixel coordinates from each camera's perspective to the canonical floor-plan coordinates. Calibrate once per camera from 4 known reference points visible in both the frame and the floor plan.

Fallback: If homography calibration is incomplete for a camera, use the simplified rule that the camera's primary zone (see Section 2) is assigned to all tracks in that camera. Flag events with `zone_confidence: "LOW"` in metadata.

Tracking and Re-Identification Strategy

Within-Camera Tracking: ByteTrack

ByteTrack is chosen over DeepSORT because:

- No Re-ID model required within a single camera.
- Handles high-confidence and low-confidence detections separately; does not discard low-confidence boxes (important for partially occluded people).
- Lightweight; runs at real-time on CPU for 5-fps streams.

Each ByteTrack `track_id` within a camera is ephemeral — it resets when a track is lost for more than `max_age` frames (set to 30 frames = 6 seconds).

Cross-Camera Re-Identification

Goal: ensure the same physical person appears as one `visitor_id` across all cameras.

Model: OSNet (`torchreid`) — lightweight ReID network; 2MB model; CPU-feasible.

Algorithm:

1. Maintain a **gallery** of `(visitor_id, feature_vector, last_seen_ts)`.
2. When a new track appears (new ByteTrack `track_id`), extract its ReID feature.
3. Compute cosine similarity against all gallery entries not seen in the last 5 minutes (to avoid cross-camera confusion with still-active tracks).
4. If `similarity ≥ 0.75`: assign same `visitor_id`; update gallery entry.
5. If `similarity < 0.75`: assign new `visitor_id`; add to gallery.
6. Gallery entries expire after 30 minutes (configurable).

Re-entry disambiguation (Section 10): a visitor who exits and re-enters within 5 minutes with $\text{similarity} \geq 0.75$ generates a `REENTRY` event, not a second `ENTRY`.

Fallback (no GPU, no ReID model): use bounding-box IoU + appearance colour histogram similarity as a weaker proxy. Flag affected visitor tokens with `reid_method`: "histogram" in metadata. This degrades cross-camera accuracy but maintains schema compliance.

Staff Filtering Strategy

Primary heuristic (recommended):

1. **Persistent track near Cash Counter**: any track whose centroid is within the `ZONE_CASH_COUNTER` polygon for $> 60\%$ of their in-store dwell time is classified as staff.
2. **Long continuous presence**: any track active for > 3 hours without an exit event is staff.
3. **Salesperson cross-reference**: the POS CSV contains `salesperson_id` and `salesperson_name`. Extract unique staff members from POS. When a transaction fires, the person near the counter at that moment (if a track exists there) is likely the salesperson. Tag that track retroactively as `is_staff=true`.

Secondary heuristic (if uniform colour is distinguishable): Train a binary classifier on a few manually labelled bounding-box crops to identify the store uniform (typically a branded apron or T-shirt). Use this as an additional signal, not the sole signal, because face blurring eliminates facial cues.

Output: All events with `is_staff=true` are stored but excluded from all customer metrics. They are preserved so reviewers can verify staff was detected, not silently discarded.

Entry / Exit Logic

Entry: A track whose centroid crosses the entry threshold line (the left wall at `ENTRY_EXIT`) from outside-to-inside.

Define the threshold as a horizontal line segment at $x = x_{\text{entry}}$ in canonical floor-plan coordinates. A crossing event fires when a track's centroid transitions from $x < x_{\text{entry}}$ to $x > x_{\text{entry}}$ across consecutive frames.

Exit: Same line, opposite direction.

Direction disambiguator:

$$\text{direction} = \text{sign}\left(\frac{\Delta x}{\Delta t}\right)$$

Average over the last 5 frames to reduce single-frame noise.

Group entry: When $n \geq 2$ detections cross the threshold within 2 seconds of each other and their centroids are < 120 px apart, treat as a group. Emit n individual `ENTRY` events with separate `visitor_ids` but the same `group_id` in metadata. This ensures individual counting while preserving the group relationship for analytics.

Camera overlap (CAM 1 / CAM 2): The entry threshold is only monitored on CAM 1 (primary entry camera). If CAM 2 FOV overlaps the entry zone, suppress `ENTRY` / `EXIT` events from CAM 2 for tracks already assigned an entry event in CAM 1 within the last 30 seconds.

Zone Dwell Logic

1. When a track enters a named zone polygon, emit `ZONE_ENTER`.

2. While the track remains in the zone, accumulate dwell time.
3. Every 30 seconds of continuous presence: emit `ZONE_DWELL` with `dwell_ms = 30000 × n` (cumulative).
4. When the track leaves the zone, emit `ZONE_EXIT` with final `dwell_ms`.

Gap tolerance: If a track disappears from a zone for < 5 seconds (occlusion by shelving), do not emit `ZONE_EXIT`; resume dwell accumulation when the track reappears. If the gap is 5–30 seconds, treat as a zone exit + re-enter. If > 30 seconds, treat as a full zone exit.

Billing Queue Logic

Queue depth estimation: Count the number of active tracks whose centroids are within the `ZONE_CASH_COUNTER` polygon (or a 150 px buffer around it) and are *not* classified as staff.

BILLING_QUEUE_JOIN: Emit when a customer track enters the billing zone *and* `queue_depth` ≥ 1 at that moment (i.e. at least one other non-staff person is already there).

BILLING_QUEUE_ABANDON: Emit when a customer track leaves the billing zone without a POS transaction being associated to their session within a 10-minute window after their exit.

POS correlation window: 5 minutes before transaction timestamp $\rightarrow$ any visitor in billing zone during that window is the buyer.

Re-entry Logic

Definition: A visitor who previously generated an `EXIT` event, re-enters the store, and whose ReID similarity to the exited `visitor_id` is ≥ 0.75 .

Time window: Re-entry is considered if the gap between `EXIT` and new `ENTRY` is < 30 minutes. Beyond 30 minutes, treat as a new visitor (gallery entry will have expired and similarity may degrade).

Event: Emit `REENTRY` (not a second `ENTRY`). The same `visitor_id` continues. The session sequence number (`session_seq`) continues from where it left off for that visitor.

API impact: The funnel endpoint de-duplicates by `visitor_id`. A visitor who re-entered still counts as 1 unique visitor, not 2. Re-entry is tracked separately for analytics (“how many customers leave and come back”).

Cross-Camera Deduplication Logic

A person visible simultaneously in CAM 1 and CAM 2 (overlap zone) must not generate two independent `ZONE_ENTER` events for the same zone.

Rule:

1. The canonical zone authority is the camera whose primary coverage includes that zone (from the camera-to-zone table in Section 2).
2. When two cameras emit a zone event for the same `visitor_id` within 5 seconds for the same zone, discard the non-authoritative one.
3. In the event merger (in `emit.py`), a per-visitor-id, per-zone event deduplication window of 5 seconds is maintained in memory.

Event Schema Definition

```

1 {
2   "event_id":      "550e8400-e29b-41d4-a716-446655440000", // UUID v4;
                        globally unique
3   "store_id":      "ST1008", // from POS
                        CSV
4   "camera_id":     "CAM_1", // CAM_1
                        through CAM_5
5   "visitor_id":    "VIS_c8a2f1", // ReID token;
                        hex suffix
6   "event_type":    "ZONE_DWELL", // see
                        catalogue below
7   "timestamp":     "2026-04-10T16:22:10+05:30", // ISO-8601 IST
8   "zone_id":       "ZONE_CASH_COUNTER", // null for
                        ENTRY/EXIT
9   "dwell_ms":      8400, // 0 for
                        instantaneous
10  "is_staff":       false, // classifier
                        output
11  "confidence":     0.91, // YOLOv8
                        detection conf
12  "metadata": {
13    "queue_depth":   null, // int; set for BILLING_QUEUE_JOIN
                        only
14    "group_id":      null, // UUID; set for group-entry members
15    "session_seq":   5, // ordinal in visitor session
16    "reid_method":   "osnet", // "osnet" | "histogram" | "bbox_iou"
17    "zone_confidence": "HIGH", // "HIGH" | "LOW" (homography quality)
18    "low_confidence": false, // true if detection conf < 0.5
19    "track_id":      1042, // internal ByteTrack ID (debug)
20    "frame_number":  9153 // source frame (debug)
21  }
22 }
```

Listing 1: Canonical event JSON — all fields required unless marked optional

Validation rules enforced at ingest:

- **event_id**: valid UUID v4; globally unique in DB.
- **store_id**: must be ST1008 (or any known store ID from config).
- **timestamp**: must be parseable ISO-8601.
- **event_type**: must be in catalogue.
- **confidence**: float in [0.0, 1.0].
- **zone_id**: required for all non-ENTRY/EXIT types; null for ENTRY/EXIT.
- **dwell_ms**: non-negative integer.

Event Type Catalogue

event_type	When emitted / semantics
ENTRY	Track crosses entry threshold inbound. Starts session. Assign new visitor_id (or continue if ReID matches).
EXIT	Track crosses entry threshold outbound. Closes session.
ZONE_ENTER	Track centroid enters a named zone polygon.
ZONE_EXIT	Track centroid leaves a named zone polygon.
ZONE_DWELL	Track has been continuously in zone for $n \times 30$ s. Emitted every 30s.
BILLING_QUEUE_JOIN	Track enters billing zone while queue_depth ≥ 1 .
BILLING_QUEUE_ABANDON	Track leaves billing zone; no POS match in next 10 min.
REENTRY	ReID matches a prior visitor_id after their EXIT.

POS Correlation and Conversion Logic

Timestamp Synthesis

```

1 from datetime import datetime, timezone, timedelta
2
3 IST = timezone(timedelta(hours=5, minutes=30))
4
5 def pos_timestamp(order_date: str, order_time: str) -> datetime:
6     # order_date: "10-04-2026", order_time: "16:55:36"
7     dt = datetime.strptime(
8         f"{order_date} {order_time}", "%d-%m-%Y %H:%M:%S"
9     )
10    return dt.replace(tzinfo=IST)

```

Listing 2: POS timestamp synthesis

Conversion Attribution

A visitor session is **converted** if:

1. At least one ZONE_ENTER or ZONE_DWELL event exists for ZONE_CASH_COUNTER with is_staff=false.
2. A POS transaction exists with timestamp in the window $[t_{\text{zone_enter}} - 30\text{s}, t_{\text{zone_enter}} + 300\text{s}]$.
3. The transaction store_id matches the session store_id.

Fallback (no visitor was detected in billing zone for that window): if the store has more POS transactions than detected billing-zone visits, the excess transactions are attributed as conversions without a matched visitor — a “dark conversion” counter is incremented in /metrics to flag detection gaps.

Conversion Rate Computation

```

1 def conversion_rate(store_id: str, window_start, window_end, db) -> float
2 :
3     total = db.execute("""
4         SELECT COUNT(DISTINCT visitor_id) FROM events
5         WHERE store_id=? AND event_type='ENTRY' AND is_staff=0
6         AND timestamp BETWEEN ? AND ?
7     """, (store_id, window_start, window_end)).fetchone()[0]
8
9     converted = db.execute("""
10        SELECT COUNT(DISTINCT e.visitor_id) FROM events e
11        JOIN pos_transactions p

```

```

11         ON p.store_id = e.store_id
12         AND p.timestamp BETWEEN
13             datetime(e.timestamp, '-30 seconds')
14             AND datetime(e.timestamp, '+300 seconds')
15         WHERE e.store_id=? AND e.zone_id='ZONE_CASH_COUNTER'
16             AND e.is_staff=0
17             AND e.timestamp BETWEEN ? AND ?
18         """, (store_id, window_start, window_end)).fetchone()[0]
19
20     return round(converted / total, 4) if total > 0 else 0.0

```

Listing 3: Conversion rate computation

API Design

POST /events/ingest

Request:

```

1 {
2   "events": [ /* array of 1..500 event objects */ ]
3 }

```

Response (200):

```

1 {
2   "accepted": 498,
3   "rejected": 2,
4   "duplicate": 1,
5   "errors": [
6     {
7       "index": 12,
8       "event_id": "...",
9       "reason": "invalid event_type: ZONE_STARE"
10    }
11  ]
12 }

```

Idempotency: Duplicate `event_ids` are silently skipped (not counted as errors). The DB has a `UNIQUE` index on `event_id`; insert-or-ignore semantics.

Partial success: Malformed events are rejected with per-event error detail. Valid events in the same batch are written.

GET /stores/{id}/metrics

Query params: `?window=today` (default) | `?from=ISO&to=ISO`

Response:

```

1 {
2   "store_id": "ST1008",
3   "window": { "from": "2026-04-10T00:00:00+05:30",
4               "to": "2026-04-10T23:59:59+05:30" },
5   "unique_visitors": 38,
6   "conversion_rate": 0.6316,
7   "avg_dwell_by_zone": {
8     "ZONE_CASH_COUNTER": 142000,
9     "ZONE_MAKEUP_UNIT": 87500

```

```

10 },
11   "queue_depth_current": 2,
12   "abandonment_rate": 0.1429,
13   "dark_conversions": 0,
14   "data_confidence": "HIGH"
15 }

```

Zero-traffic handling: If no events exist for the window, return all counters as 0 and `data_confidence`: "NO_DATA". Never return null or a 5xx.

GET /stores/{id}/funnel

Response:

```

1 {
2   "store_id": "ST1008",
3   "funnel": [
4     { "stage": "ENTRY",          "count": 38, "drop_pct": 0.0   },
5     { "stage": "ZONE_VISIT",     "count": 34, "drop_pct": 10.53  },
6     { "stage": "BILLING_QUEUE",  "count": 26, "drop_pct": 23.53  },
7     { "stage": "PURCHASE",       "count": 24, "drop_pct": 7.69   }
8   ],
9   "session_unit": true,
10  "reentry_excluded": true
11 }

```

Session deduplication: Each `visitor_id` counts once per funnel stage regardless of how many events they generated. Re-entries do not create a second session entry.

GET /stores/{id}/heatmap

Response:

```

1 {
2   "store_id": "ST1008",
3   "zones": [
4     {
5       "zone_id": "ZONE_MAKEUP_UNIT",
6       "visit_count": 22,
7       "avg_dwell_ms": 87500,
8       "normalised_score": 100,
9       "data_confidence": "HIGH"
10    },
11    {
12      "zone_id": "ZONE_FRAGRANCE",
13      "visit_count": 3,
14      "avg_dwell_ms": 12000,
15      "normalised_score": 14,
16      "data_confidence": "LOW"
17    }
18  ]
19 }

```

`data_confidence`: "LOW" if fewer than 20 sessions contributed to that zone's data in the window.

Normalisation: `normalised_score = round(visit_count / max_visit_count * 100)`.

GET /stores/{id}/anomalies

Response:

```

1 {
2   "store_id": "ST1008",
3   "anomalies": [
4     {
5       "anomaly_id": "uuid",
6       "type": "BILLING_QUEUE_SPIKE",
7       "severity": "WARN",
8       "detected_at": "2026-04-10T19:05:00+05:30",
9       "description": "Queue depth reached 5 (threshold: 3)",
10      "suggested_action": "Deploy additional staff to billing counter"
11    },
12    {
13      "anomaly_id": "uuid",
14      "type": "DEAD_ZONE",
15      "severity": "INFO",
16      "zone_id": "ZONE_FRAGRANCE",
17      "detected_at": "2026-04-10T15:00:00+05:30",
18      "description": "No zone visits in last 30 minutes",
19      "suggested_action": "Check zone display; consider promotional signage"
20    }
21  ]
22 }

```

Anomaly detection rules:

Type	Severity	Rule
BILLING_QUEUE_SPIKE	WARN/CRIT	Queue_depth > 3 for > 2 minutes; CRIT if > 5 for > 5 min
CONVERSION_DROP	WARN	conversion rate < 70% of 7-day average (use single-day average if no history)
DEAD_ZONE	INFO	No ZONE_ENTER for a named zone in 30 minutes
EMPTY_STORE	INFO	No active visitor tracks for > 10 minutes
STALE_FEED	WARN	Last event for a camera > 10 minutes ago during store hours

GET /health

```

1 {
2   "status": "ok",
3   "uptime_seconds": 3820,
4   "db_connected": true,
5   "stores": {
6     "ST1008": {
7       "last_event_ts": "2026-04-10T19:07:33+05:30",
8       "cameras": {
9         "CAM_1": { "last_event_ts": "...", "stale": false },
10        "CAM_3": { "last_event_ts": "2026-04-10T18:50:00+05:30",
11                  "stale": true, "reason": "STALE_FEED" }
12      }
13    }
14  }
15 }

```


Storage Choice and Rationale

Choice: SQLite (primary), in-memory dict cache (hot metrics)

Option	Trade-off
SQLite	Zero setup; <code>docker compose</code> up just works; sufficient for single-store, 20-minute clip volume; ACID guarantees idempotency; file-based means data survives container restart
PostgreSQL	Required at 40-store real-time scale; overkill for this challenge and adds setup complexity; rejected
Redis	Useful for queue-depth real-time counter; add as optional compose service if dashboard needs sub-second latency

Schema (SQLite):

```

1 CREATE TABLE events (
2   event_id      TEXT PRIMARY KEY,
3   store_id      TEXT NOT NULL,
4   camera_id     TEXT NOT NULL,
5   visitor_id    TEXT NOT NULL,
6   event_type    TEXT NOT NULL,
7   timestamp     TEXT NOT NULL,      -- ISO-8601 string
8   zone_id       TEXT,
9   dwell_ms      INTEGER NOT NULL DEFAULT 0,
10  is_staff       INTEGER NOT NULL DEFAULT 0,  -- 0/1
11  confidence     REAL NOT NULL,
12  metadata_json  TEXT,                -- JSON blob for metadata fields
13  ingested_at    TEXT NOT NULL        -- server insert time
14 );
15
16 CREATE INDEX idx_store_ts ON events(store_id, timestamp);
17 CREATE INDEX idx_visitor ON events(visitor_id);
18 CREATE INDEX idx_zone    ON events(zone_id, timestamp);
19
20 CREATE TABLE pos_transactions (
21   order_id      INTEGER PRIMARY KEY,
22   store_id      TEXT NOT NULL,
23   timestamp     TEXT NOT NULL,
24   total_amount  REAL,
25   salesperson_id INTEGER,
26   dep_name      TEXT,
27   brand_name    TEXT,
28   matched_visitor TEXT              -- populated after POS correlation job
29 );

```

Listing 4: SQLite DDL

What is cached vs. computed live:

Metric	Strategy	Reason
Queue depth current	Live	Must reflect latest event; no stale cache
Unique visitor count	Live	Accuracy; dataset small enough
Conversion rate	Live	Changes with each new POS or billing event
Heatmap scores	Cache 60s	Computationally heavier; acceptable 60s lag
Anomaly list	Cache 30s	Evaluated on a background thread every 30s
7-day avg conversion	Pre-computed	Requires historical data; computed on startup

Idempotency Strategy

1. `event_id` is a UUID v4 generated by the detection pipeline.
2. The `events` table has `event_id` TEXT PRIMARY KEY.
3. Ingest uses `INSERT OR IGNORE INTO events ....`
4. The ingest response reports duplicate count separately (not as an error).
5. Sending the same 500-event batch twice produces identical DB state.
6. Idempotency test in `tests/test_ingestion.py` verifies this explicitly.

Logging and Observability

Structured log format (JSON lines to stdout):

```

1 {
2   "ts": "2026-04-10T19:07:33Z",
3   "level": "INFO",
4   "trace_id": "abc123",
5   "store_id": "ST1008",
6   "endpoint": "POST /events/ingest",
7   "latency_ms": 42,
8   "event_count": 250,
9   "status_code": 200
10 }
```

Required fields per endpoint:

Field	Notes
<code>trace_id</code>	UUID per request; return in X-Trace-ID response header
<code>store_id</code>	Extracted from path param
<code>endpoint</code>	METHOD /path
<code>latency_ms</code>	Measured via FastAPI middleware
<code>event_count</code>	For ingest only
<code>status_code</code>	HTTP status

Health metrics exposed at `/health` for on-call engineers.

No raw stack traces in responses: all unhandled exceptions map to a 500 with body `{"error": "internal_error", "trace_id": "..."}.`

Error Handling and Graceful Degradation

Failure mode	HTTP	Response body
DB unavailable	503	<code>{"error": "db_unavailable", "trace_id": "..."}.</code>
Unknown <code>store_id</code>	404	<code>{"error": "store_not_found"}</code>
Malformed event batch	207	Partial success body (Section 15)
Empty store (zero events)	200	All counters 0; <code>data_confidence: "NO_DATA"</code>
All-staff clip (no customers)	200	<code>unique_visitors: 0, conversion_rate: 0.0</code>
Zero purchases	200	<code>conversion_rate: 0.0</code> (not null, not error)
Invalid query params	422	Pydantic validation error (FastAPI default)

Test Strategy and Edge Cases

Test Structure

Test file	Coverage
test_pipeline.py	Event schema validation; entry/exit direction; group entry count; staff flag; confidence passthrough
test_ingestion.py	Idempotency (double-post same batch); partial failure; event_id uniqueness; 500-event batch limit
test_metrics.py	Conversion rate formula; zero-purchase store; all-staff clip; zero-traffic window
test_funnel.py	Re-entry de-duplication; session counting; drop-off % math
test_anomalies.py	Queue spike trigger; dead zone trigger; stale feed detection
test_health.py	DB-down 503; stale camera warning

Target: statement coverage > 70% (pytest -cov).

Key Edge Cases and How to Test

1. **Empty store:** ingest events with no ENTRY type → expect `unique_visitors: 0`, not null.
2. **All-staff clip:** ingest 50 events all with `is_staff=true` → `unique_visitors: 0`.
3. **Zero purchases:** no POS rows for window → `conversion_rate: 0.0`.
4. **Re-entry in funnel:** two ENTRY events for same visitor_id → funnel counts visitor once; re-entry counter increments.
5. **Group entry:** 3 ENTRY events within 2s same group_id → `unique_visitors` increments by 3.
6. **Idempotency:** POST same 100-event payload twice → DB row count unchanged; response shows `duplicate: 100` on second call.
7. **Low confidence:** events with `confidence: 0.31` are stored and appear in metrics; not silently dropped.
8. **Queue abandon:** BILLING_QUEUE_JOIN followed by exit, no POS match in 10 min → BILLING_QUEUE_ABANDON event emitted.

Docker Compose Layout

```

1 version: "3.9"
2 services:
3   api:
4     build: ./app
5     ports:
6       - "8000:8000"
7     volumes:
8       - ./data:/data          # SQLite db lives here
9       - ./events:/events      # .jsonl event files from pipeline
10    environment:
11      - DB_PATH=/data/store_intelligence.db
12      - LOG_LEVEL=INFO
13    healthcheck:
14      test: ["CMD", "curl", "-f", "http://localhost:8000/health"]
15      interval: 10s
16      timeout: 5s
17      retries: 3
18
19    dashboard:
20      build: ./dashboard

```

```

21     ports:
22         - "8501:8501"                # or terminal-only: no port needed
23     depends_on:
24         api:
25             condition: service_healthy
26     environment:
27         - API_BASE=http://api:8000

```

Listing 5: docker-compose.yml

Note: The detection pipeline (pipeline/) is not a compose service — it runs as a one-shot script on the host (or in a container if GPU is available) and writes events to `./events/events.jsonl`, which is then POSTed to the API. This separation keeps the compose stack simple and always runnable without a GPU.

Repository Structure

```

store-intelligence/
|---- pipeline/
|   |---- detect.py           # YOLOv8n inference; 5-fps frame extraction
|   |---- tracker.py         # ByteTrack wrapper; cross-camera Re-ID
|   |---- staff.py           # Staff classification heuristics
|   |---- emit.py            # Sessionisation; event-type FSM; JSON-L writer
|   |---- config/
|   |   |---- zones.json     # Zone polygons in canonical floor coords
|   |   \---- cameras.json   # Camera metadata; homography matrices
|   \---- run.sh             # Process all 5 clips; POST to /events/ingest
|---- app/
|   |---- main.py            # FastAPI app; lifespan; routers
|   |---- models.py          # Pydantic schemas: Event, IngestRequest, etc.
|   |---- ingestion.py       # Validate; dedup; batch-insert
|   |---- metrics.py         # /metrics endpoint logic
|   |---- funnel.py          # /funnel logic; session dedup
|   |---- heatmap.py         # /heatmap normalisation
|   |---- anomalies.py       # Rule-based anomaly detection
|   |---- health.py          # /health; stale-feed check
|   |---- db.py              # SQLite connection; DDL migrations
|   |---- pos.py             # POS CSV loader; correlation logic
|   \---- Dockerfile
|---- dashboard/
|   |---- app.py             # Rich terminal dashboard OR lightweight web UI
|   \---- Dockerfile
|---- tests/
|   |---- test_pipeline.py   # PROMPT: <AI prompt> / CHANGES MADE: <...>
|   |---- test_ingestion.py  # PROMPT: ...
|   |---- test_metrics.py    # PROMPT: ...
|   |---- test_funnel.py     # PROMPT: ...
|   \---- test_anomalies.py  # PROMPT: ...
|---- docs/
|   |---- DESIGN.md
|   \---- CHOICES.md
|---- data/                  # SQLite DB (git-ignored)
|---- events/                # Generated .jsonl files (git-ignored)
|---- footage/               # CCTV clips (git-ignored; symlink or .gitignore)

```

```
|---- docker-compose.yml
|---- README.md
\---- .env.example
```

README Runbook

```
1 # 1. Clone and enter
2 git clone <repo-url> store-intelligence && cd store-intelligence
3
4 # 2. Place footage clips
5 cp /path/to/footage/*.mp4 footage/
6
7 # 3. Start the API and dashboard
8 docker compose up -d
9
10 # 4. Run the detection pipeline against all 5 clips
11 bash pipeline/run.sh \
12   --clip-dir footage/ \
13   --clip-start "2026-04-10T10:00:00+05:30" \
14   --api http://localhost:8000
15
16 # 5. Query metrics
17 curl http://localhost:8000/stores/ST1008/metrics
```

Listing 6: 5-command setup

Optional — load POS transactions manually:

```
1 curl -X POST http://localhost:8000/admin/load-pos \
2   --form "file=@Brigade_Bangalore_10_April_26.csv"
```

Dashboard URL: <http://localhost:8501> (or terminal: `docker compose logs -f dashboard`)

DESIGN.md Content Plan

Required sections (> 250 words; must be non-trivial):

1. **System overview** — one-page architecture diagram (ASCII or Mermaid) + paragraph on design philosophy (batch-first, then real-time).
2. **Detection layer** — model choice; why 5 fps; how ByteTrack handles occlusion; where it fails and how confidence propagation mitigates it.
3. **Sessionisation logic** — explain the finite state machine per `visitor_id`; entry → zone events → exit; how re-entry extends the same session.
4. **POS correlation** — no customer ID in POS data; explain the time-window attribution approach; acknowledge its limitations (two customers at counter simultaneously creates ambiguity).
5. **AI-Assisted Decisions** (required section):
 - Decision 1: Chose ByteTrack over DeepSORT after querying an LLM about occlusion handling trade-offs; agreed with AI recommendation.
 - Decision 2: LLM suggested using a VLM (GPT-4V) for zone classification. Overrode this in favour of polygon-based rules; reasoning: VLM latency and cost are prohibitive for frame-level inference on a 20-minute clip.
 - Decision 3: LLM suggested Redis for real-time queue depth. Chose SQLite-only for challenge scope; acknowledged the production upgrade path.

6. **Known limitations** — homography calibration is manual; face-blurred footage removes a potential ReID signal; single-day POS data makes 7-day anomaly comparison impossible without synthetic history.

CHOICES.md Content Plan

Three decisions with full reasoning (> 250 words total):

1. Detection model: YOLOv8n + ByteTrack

- Options considered: YOLOv8n, YOLOv8s, RT-DETR, MediaPipe.
- What AI suggested: YOLOv8s for better occlusion accuracy.
- What was chosen and why: YOLOv8n because it runs on CPU without a GPU (challenge does not guarantee GPU); acceptable accuracy at retail density; 5-fps downsampling compensates for lower per-frame accuracy with temporal consistency.
- If GPU is available at runtime: swap to YOLOv8s via config flag; no code change required.

2. Event schema design rationale

- Why **confidence** is never suppressed: reviewers explicitly check for this; a dropped low-confidence event loses information that could surface partial occlusion patterns.
- Why **session_seq** is in metadata: allows replay/debugging of exact visitor journey without re-querying the DB.
- Why **dwell_ms=0** for instantaneous events: avoids null handling downstream; 0 is semantically accurate for ENTRY/EXIT.
- Deviation from brief: **sku_zone** field from the brief example schema is not included because the actual floor-plan has brand zones, not SKU zones; replaced with **zone_confidence**.

3. API architecture: SQLite + FastAPI

- Options: SQLite, PostgreSQL, TimescaleDB.
- What AI suggested: PostgreSQL for production readiness.
- What was chosen: SQLite because the acceptance gate requires **docker compose up** with zero manual steps; PostgreSQL adds a service dependency and init scripts. SQLite is sufficient for single-store, single-day volume (~5,000 events total).
- Upgrade path documented: swap **db.py** connection string; DDL is ANSI-compatible.

Prompt Blocks for Test Files

Each test file should begin with a comment block in the following format:

```

1 # PROMPT:
2 #   "Write pytest tests for an event ingestion endpoint.
3 #   The endpoint is POST /events/ingest. It must be idempotent
4 #   by event_id. Test: (1) normal batch of 50 events accepted,
5 #   (2) duplicate batch returns duplicate count not error,
6 #   (3) malformed event in batch causes partial success,
7 #   (4) batch of 501 events rejected with 422.
8 #   Use FastAPI TestClient and an in-memory SQLite DB fixture."
9 #
10 # CHANGES MADE:
11 #   - AI generated a fixture using pytest tmp_path; replaced with
12 #   an in-memory :memory: SQLite DB for speed.
13 #   - AI did not handle the 501-event boundary; added manually.
14 #   - Renamed test_ingest_duplicates to test_ingest_idempotent
15 #   for clarity.
```

Listing 7: Test file prompt block template

Live Dashboard / Real-Time Display

Minimal (terminal, no bonus points risk)

Use the `rich` library to render a live table refreshed every 5 seconds:

```

1 from rich.live import Live
2 from rich.table import Table
3 import httpx, time
4
5 def make_table(metrics):
6     t = Table(title="ST1008 Brigade Road --- Live Metrics")
7     t.add_column("Metric"); t.add_column("Value")
8     t.add_row("Unique Visitors", str(metrics["unique_visitors"]))
9     t.add_row("Conversion Rate", f"{metrics['conversion_rate']:.1%}")
10    t.add_row("Queue Depth", str(metrics["queue_depth_current"]))
11    t.add_row("Abandonment Rate", f"{metrics['abandonment_rate']:.1%}")
12    return t
13
14 with Live(refresh_per_second=0.2) as live:
15     while True:
16         r = httpx.get("http://localhost:8000/stores/ST1008/metrics")
17         live.update(make_table(r.json()))
18         time.sleep(5)

```

Bonus-point web UI

Serve a minimal `index.html` that polls `/stores/ST1008/metrics` every 3 seconds and renders a bar chart (Chart.js from CDN). Mount as a static file in FastAPI. Expose at `/dashboard`.

Alternatively, implement a `GET /stores/{id}/stream` Server-Sent Events endpoint that pushes metric updates whenever new events are ingested; connect from the HTML page with `EventSource`.

Concrete Build Order

1. Step 1 — Schema and DB

Implement `models.py` (Pydantic), `db.py` (DDL), `POST /events/ingest` (validate + insert-or-ignore). *Validate*: POST 10 hand-crafted events; confirm idempotency. *Fail mode to test*: DB file missing → 503.

2. Step 2 — POS loader and metrics

Load POS CSV; implement `/stores/{id}/metrics`. *Validate*: With 0 events, return all zeros. With events + POS, check conversion rate is a float in `[0,1]`.

3. Step 3 — Funnel and heatmap

Implement session deduplication; funnel stages; heatmap normalisation. *Validate*: Re-entry test case; all-staff test case.

4. Step 4 — Anomalies and health

Add anomaly rules; add `/health` stale-feed check.

5. Step 5 — Docker and README

Write Dockerfile for API; write `docker-compose.yml`; confirm `docker compose up && curl /health` passes.

6. Step 6 — Detection pipeline (CAM 1 first)

Install YOLOv8 + ByteTrack; run on CAM 1.mp4; confirm ENTRY/EXIT events are emitted with correct timestamps. *Validate*: Visual inspection of bounding boxes + entry crossings.

7. Step 7 — ReID and cross-camera

Add OSNet; cross-camera dedup; staff detection heuristic. *Validate*: Same person walking CAM 1 → CAM 2 produces one `visitor_id`, not two.

8. Step 8 — Full pipeline + zone classification

Define zone polygons in `zones.json`; wire homography; emit `ZONE_ENTER` / `ZONE_DWELL` / billing events.

9. Step 9 — Tests and coverage

Write all test files with prompt blocks. Run `pytest -cov`.

10. Step 10 — Dashboard, DESIGN.md, CHOICES.md

Minimal shippable version (if time is short): Steps 1–5 + Step 6 (CAM 1 only) give a passing acceptance gate. The API works; metrics return real values; docker compose up runs. Parts D and E are optional; skip if time is short.

Validation Checklist

- ☐ `docker compose up` starts without errors on a clean machine.
- ☐ `GET /health` returns `{"status": "ok"}` within 30s of startup.
- ☐ `POST /events/ingest` with the 200-event sample returns 200.
- ☐ Sending same 200 events twice: second call shows `duplicate: 200`.
- ☐ `GET /stores/ST1008/metrics` returns valid JSON with numeric fields.
- ☐ `GET /stores/ST1008/metrics` with no events returns all zeros, not 5xx.
- ☐ `GET /stores/ST1008/funnel` shows 4 stages with `drop_pct` values.
- ☐ `GET /stores/ST1008/anomalies` returns valid JSON array (may be empty).
- ☐ `GET /stores/ST1008/heatmap` shows all zone IDs with normalised scores.
- ☐ `pytest -cov` reports $> 70\%$ statement coverage.
- ☐ `DESIGN.md` exists, > 250 words, includes *AI-Assisted Decisions* section.
- ☐ `CHOICES.md` exists, > 250 words, covers 3 decisions.
- ☐ Each test file has `# PROMPT:` and `# CHANGES MADE:` header.
- ☐ All emitted events validate against Pydantic schema (no extra fields, no missing required fields).
- ☐ Staff events are stored with `is_staff=1`; excluded from customer metric queries.
- ☐ Low-confidence events (`confidence < 0.5`) are present in DB, not dropped.
- ☐ Re-entry produces `REENTRY` event; visitor counted once in funnel.
- ☐ Group entry produces n separate `ENTRY` events, not 1.

Assumptions and Open Questions

Confirmed Facts

- Store: single store, ST1008, Brigade Road, Bangalore.
- Video: 5 cameras (CAM 1–5); no footage from any other store.
- POS: 101 line items, 24 unique orders, single date 10-Apr-2026.
- POS time range: 12:15–21:39 IST.
- POS schema: 39 columns including `salesperson_id` (staff signal).
- Store layout: single PNG floor-plan with 23 named zones identifiable.
- Zones include dedicated Cash Counter (billing) and F.O.H. (main floor).
- Brand zones map to POS `brand_name` and `dep_name` columns.

Assumptions Made

- **A-1:** Camera-to-zone mapping inferred from file sizes; must be confirmed by watching first 30 seconds of each clip.
- **A-2:** Clip start time assumed 10:00:00 IST on 10-Apr-2026.
- **A-3:** Clips do not span midnight; all timestamps are same calendar date.

- **A-4:** All 5 cameras belong to the same store (ST1008).
- **A-5:** Face blurring is the only anonymisation applied; body and clothing are visible, enabling ReID.
- **A-6:** POS timestamps are IST (not UTC) based on the local store context.
- **A-7:** No `sample_events.jsonl` or `assertions.py` are available; schema compliance tested against the brief specification only.
- **A-8:** The “Revised” / “Current” cells in the XLSX are revision markers, not zone data.
- **A-9:** 7-day average conversion rate for anomaly detection uses synthetic baseline (0.55) since only one day of POS data is available.

What Still Needs Confirmation

1. **Clip start time:** watch each clip and check if any environmental cues (daylight, clock visible, customer density) confirm the recording window. Cross-reference file modification timestamps.
2. **Camera angles:** watch the first 60 seconds of each clip to confirm the camera-to-zone mapping in Section 2.
3. **Staff count:** how many unique salesperson IDs in POS? Are all on-floor staff transacting, or are some floor staff non-transacting?
4. **CAM 4 and CAM 5 zones:** are they truly zone-specific cameras or do they cover the full floor from different angles?
5. **Entry/exit direction in CAM 1:** confirm which horizontal direction is entry vs. exit by watching the first customer arrival.
6. **Homography feasibility:** are there at least 4 identifiable reference points visible in each camera frame that also appear on the floor plan?
7. **GPU availability at runtime:** determines whether YOLOv8n (CPU) or YOLOv8s/m (GPU) should be used; pipeline should auto-detect via `torch.cuda.is_available()`.
8. **Customer name as identity proxy:** the POS CSV has `customer_name` (not anonymised); could be used to count unique customers more accurately — but mixing PII with video ReID raises ethical concerns; treat as out-of-scope unless explicitly permitted.

End of Master Implementation Blueprint